Answer Key & Rubric — Week 9, Class 1

Centralized Multi-Agent Patterns

> **Instructor copy.** Model answers and a point-based rubric for the Week 9, Class 1 quiz. Total: **100 points**. Answers are grounded in Module 9 — Centralized Multi-Agent Patterns; section references are given so you can point students back to the source.

How to grade

Each question is scored against the criteria below. These are short-answer prompts (2–4 sentences), so award **partial credit** generously for reasoning that lands on the right idea in different words — the quiz explicitly warns that *definitions alone will not score*, so a correct restatement with no application should top out at roughly half the question's points. Deduct for answers that only name patterns without justifying the fit, that confuse Orchestrator-Worker with Supervisor-Router (the exact look-alike this module exists to separate), or that justify adding an agent without naming a specific failure mode.

Question 1 — Match the pattern to the task's control-flow shape (34 points)

Assign the right centralized pattern to a genuinely fixed sequence and to a task whose next sub-task depends unpredictably on prior results, then explain the flexibility-for-predictability trade of centralized control.

Model answer. The fixed sequence (extract → transform → summarize → cite) is a **Sequential Pipeline** — the order is wired in by you, in code, with no decomposition or routing decision at run time (Module 9, *Sequential Pipeline — a fixed chain of transformations*). The task whose next sub-task depends unpredictably on what the last one produced is a **Supervisor-Router**: the supervisor inspects the current state and decides which single agent handles the next turn, re-deciding after each turn given what just happened — ReAct at the system level (Module 9, *Supervisor-Router — route the next turn*). Centralized control trades

flexibility for predictability and debuggability because a single named component owns the control flow, so there is *one place* that decided the order and you can inspect its reasoning when the system misbehaves (Module 9, *Centralized means a controller owns the control flow*). The pipeline is the most predictable and debuggable of the four precisely because it is the least flexible — the order can never adapt at run time — while the supervisor buys mid-task adaptivity at the cost of a route that only emerges turn by turn.

Rubric.

Criterion	Points
Fixed sequence → **Sequential Pipeline**, with reason (order fixed in code, no runtime decision)	10
Unpredictable next step → **Supervisor-Router**, with reason (routes each turn from latest state)	10
Explains the flexibility-vs-predictability/debuggability trade — one named controller = one place decided the order	14

Common deductions: −5 if the student names the two patterns correctly but never explains *why* centralized control is more predictable/debuggable. Accept "Orchestrator-Worker" for the second task **only** if the student explicitly argues the sub-tasks are knowable up front — but the "depends unpredictably" wording points to the supervisor, so an unjustified orchestrator answer loses the second criterion.

Question 2 — The coordination tax and the fourth worker (33 points)

Say what benefit would have to materialize to justify a fourth worker against the coordination tax, and name a failure mode you would be inviting.

Model answer. The coordination tax means every added agent multiplies delegation calls, latency, tokens, failure modes, and debugging difficulty, so a fourth worker only pays for itself if it delivers one of the honest reasons on the escalation list — for example genuine **parallelism** (an independent sub-task that now runs at once, cutting wall-clock latency), a **conflicting-prompts** split (the fourth job needs a constitution the other three cannot share), **independent verification**, or **domain-specific tooling** the other workers should not carry (Module 9, *Where this module sits*; *Orchestrator-Worker — decompose, dispatch, aggregate*). The key point is that you do not earn the fourth agent by wanting one — you earn it by naming

the specific way three workers failed. The failure mode I invite is **bad decomposition**: adding a worker forces the orchestrator to re-carve the task into four pieces that may now overlap or miss part of the goal, and the aggregation step can paper over a worker that silently failed or returned a confident wrong result (Module 9, *What Orchestrator-Worker costs*). Push it further and it becomes **over-decomposition** — more, smaller sub-tasks than the problem needs — which is severe enough to be a named anti-pattern in Module 12.

Rubric.

Criterion	Points
Names a *specific* benefit that would justify the fourth worker (parallelism, conflicting prompts, independent verification, or domain-specific tooling)	12
Ties the justification to the escalation principle — earn the agent by a named failure, not by wanting one	9
Names a genuine failure mode being invited (bad/over-decomposition, lossy aggregation, cascading failure) tied to the added agent	12

Common deductions: −5 if the "benefit" is vague ("it would be faster" / "it could do more") rather than one of the named escalation reasons. −4 if the failure named is generic ("it might be wrong") rather than a characteristic coordination-tax failure.

Question 3 — Shared MCP tools across workers (33 points)

Explain why exposing overlapping capabilities through a shared MCP tool protocol beats hand-wiring tools into each agent, and what a FastMCP server buys you when swapping or versioning a tool.

Model answer. In a centralized system where several workers need overlapping capabilities, a **shared MCP tool protocol** means each capability is defined once behind a standard interface and every worker calls it the same way, instead of the same tool being hand-wired — and separately maintained — inside each agent (Module 9, *Orchestrator-Worker* — each worker has its own tools; Primers 19–20 on MCP/FastMCP). That matters because it removes duplicated, drifting tool code: one authoritative implementation serves all workers, so a fix or capability improvement lands everywhere at once rather than being re-applied per agent. A **FastMCP server** buys you a clean seam for change: because workers depend on the tool's *protocol interface* rather than its internals, you can swap the implementation or ship a new

version behind the same interface and none of the workers need rewriting — the coordination tax of a multi-worker system is exactly what you do not want to pay again every time a tool changes. This keeps the control-flow story (one controller owns the flow) separate from the capability story (tools are shared infrastructure), which is what makes a centralized multi-agent system maintainable as it grows.

Rubric.

Criterion	Points
Explains the shared-protocol benefit: define a capability once behind a standard interface vs. duplicating/hand-wiring it into every worker	13
Explains what FastMCP buys for swap/version: workers depend on the interface, so the implementation changes without rewriting every worker	12
Connects it back to the multi-agent context — avoids re-paying coordination/maintenance cost per worker; keeps tools as shared infrastructure	8

Common deductions: −5 if the answer describes MCP generically without engaging the *multi-worker overlapping-capabilities* setting the question specifies. −4 if the versioning point is asserted but the interface-vs-implementation reason is missing.

Grounding. All answers trace to Module 9: the definition of centralized control (*Centralized means a controller owns the control flow*), the four patterns (*Orchestrator-Worker*, *Hierarchy*, *Supervisor-Router*, *Sequential Pipeline*), the coordination tax and escalation list (*Where this module sits*), and each pattern's cost/failure tables. Question 3 also draws on Primers 19–20 (MCP/FastMCP). Students who cite these anchors by name should be rewarded.